

CS 230 Distributed Computer Systems

Homework 5

1 Batch Initialization

At first, we applied the `k` values by suggested values of `{5, 10, 100}`. We can construct the `k` value array in the "`_main_`" function as follows:

```
processor_batches = [5, 10, 100]
```

2 Batch Unbalancing

Then, we can assign random distributed load unit to each processor batch within the suggested interval of `[10, 1000]` to unbalance the work loads as follows:

```
for batch in processor_batches:  
    loads = [random.randint(10, 1000) for i in range(batch)]
```

3 Load Balancing Strategy Implementation

3.1 Balance Checking

We can implement a balance checking function that check if the work loads are globally balanced in every cycle with an argument of work loads:

```
def balance_check_global.loads):
```

Then, we can calculate the global average load value. It will be used to keep all the load of processors in a balanced value range. Since the actual average value may not be an integer, we can record the average value in a float point value as well.

```
avg = sum.loads) / len.loads)  
avg_f = float(sum.loads)) / len.loads)
```

Then set a boundary around the average load value, every processor's load should fall within the boundary, otherwise we deem it as unbalanced:

```

boundary = avg = avg_f ? [avg, avg] : [avg, avg + 1]
for load in loads:
    if load not in boundary:
        return False
return True

```

3.2 Balancing function Implementation

Since we are balancing between three consecutive nodes, so we need these three arguments in the function. Besides, the work load should also be included:

```

def balance_operation(loads, left, mid, right):

```

In this function, we should at first get the average load value of the three processors, the balancing operation will be based on it:

```

    avg = (loads[left] + load[mid] + load[right]) / 3

```

With the average(balanced) value, we can reassign the three processors with adjusted load value as follows:

```

    if avg * 3 % 3 == 1:
        loads[left], loads[mid], loads[right] =
            (avg, avg + 1, avg)
    elif avg * 3 % 3 == 2:
        loads[left], loads[mid], loads[right] =
            (avg, avg + 1, avg + 1)
    else:
        loads[left], loads[mid], loads[right] =
            (avg, avg, avg)

```

3.3 Balancing Strategy Implementation

With the balancing function and balance checking function provided above, we can implement the balance strategy as follows:

```

while !balance_check(loads) and cycle_num < 1000000:
    cycle_num += 1
    for i in range(size):
        balance_operation(loads, (i - 1) % size,
                           i, (i + 1) % size)

```

4 Load Activity Scheduling

Next, we will be working on scheduling load activity for each processor using a random number generator with uniform distribution. Following the suggestion in instruction, we set the range of random time interval to [100, 1000] as follows:

```

balancing_cycles = [random.randint(100, 1001)
                    for i in range (batch_size)]

```

We need a cycle number counter to record the cycle time:

```

cycle_num = 0;

```

So the scheduling process can be implemented as follows:

```

while !balance_check(loads):
    cycle_num += 1
    for i in range(batch_size):
        if cycle_num >= balancing_cycles[i]:
            balancing_cycles[i] += random.randint(100, 1001)
            balance_operation(loads, (i - 1) % size,
                              i, (i + 1) % size)

```

In the above function, we use check if the current `cycle_num` is larger than the time window. If it is true, it means that it is the time for the current processor to perform a local balance. If we encapsulate it into a function, when the balancing loop is terminated, we can return the cycle number.

5 Time Limit Imposition

In the scheduling process, in case of infinite loop, we can set the max cycle number to 1000000 times. When the cycle number reach this limit, the schedule process will be terminated.

With this limitation on the maximum cycle number, we can revise the previous scheduling process as follows:

```
while !balance_check(loads) and cycle_num < 1000000:  
    cycle_num += 1  
    for i in range(batch_size):  
        if cycle_num >= balancing_cycles[i]:  
            balancing_cycles[i] += random.randint(100, 1001)  
            balance_operation(loads, (i - 1) % size,  
                             i, (i + 1) % size)
```

6 Initial Experiment Result and Analysis

After executing the program, we can get the average balance cycle time of 5, 10, and 100 batch processors as follows:

batch	cycles
5	1000000
10	1000000
100	1000000

It shows that every batch reaches the limit of 1000000 cycles, and fail to balance the loads. Obviously, we have to choose another way to implement the program.

7 Alternative Balancing Strategy

To re-implement the balancing algorithm, we should re-implement the function that checks if the load unit is balanced. Previously, we check the balancing by checking the global balance. This method has its drawbacks. Since it is only globally balanced, we only compare the load of each processor with a global average value, which cannot ensure the balance between the neighboring processors. So we should check the balance locally, by set of three processors.

As for the first and last processor in the batch, they only have one neighboring unit. However, we can regard the last processor the left neighbor processor of the first processor, vice versa, by mode them by the size of the batch. And we can get the size of the batch by:

```
batch_size = len(loads)
```

With the batch size, we can calculate and judge the balance condition of each processor by iterating the whole batch, then return a boolean value as follows:

```

for i in range(batch_size):
    return abs(loads[i] - loads[(i - 1) % batch_size]) > 1 or
           abs(loads[i] - loads[(i + 1) % batch_size]) > 1 ?
False : True

```

We can encapsulate the above loop in a balance checking function as follows:

```

def balance_check(loads):

```

8 Final Experiment Result and Analysis

After executing the program, we can get the average balance cycle time of 5, 10, and 100 batch processors as follows:

batch	cycles
5	202153.3
10	106471.7
100	63022.2

Now, let's dig into the terminal log to see what has happened.

In the logs of 5 batch, we can see the result of 10 trials. In the log, we can see that there are two logs show that there are 1000000 cycles, which is the maximum cycle number, which means it fails to converge to be balanced. When these two logs are excluded, the minimum cycle number is 1820, while the maximum cycle number is 3406. In most cases, the balancing algorithm works fine.

```

2509 2542 3341 1820 2014 1000000 3406 1000000 2621 3280

```

In the logs of 10 batch, we can see that there is one log shows that there are 1000000 cycles, which is the maximum cycle number, which means it fails to converge to be balanced. When these one log is excluded, the minimum cycle number is 5802, while the maximum cycle number is 9191.

```

5802 6955 7676 7080 9191 1000000 7841 7336 6673 6190

```

In the logs of 100 batch, there is nothing abnormal. The minimum cycle number is 58587, while the maximum cycle number is 68602. We can infer that in most cases, the balancing algorithm works fine.

```

58587 64780 68602 61759 64168 59520 62011 68487 61419 60889

```

From this result, we can see that our strategy actually works.